

---

# Projet Développement | CSC3502

## Rapport de projet : Polaris

---

*Groupe*

Chadi Ait Ekioui

Emma Mell

Ryane Seri

Beryl Sperelakis-Beedham

*Professeur encadrant*

Eric Lallet

17 Mai 2025

# Table des matières

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>1</b>  |
| 1.1      | Objectifs . . . . .                                                 | 1         |
| 1.2      | Contexte . . . . .                                                  | 1         |
| 1.3      | Plan & Recettes . . . . .                                           | 1         |
| 1.3.1    | Reconnaissance Automatique de Constellations . . . . .              | 2         |
| 1.3.2    | Interface . . . . .                                                 | 2         |
| 1.3.3    | Diagramme de classes . . . . .                                      | 3         |
| 1.4      | Choix des Technologies . . . . .                                    | 3         |
| 1.5      | Gestion de projet . . . . .                                         | 4         |
| <b>2</b> | <b>Conception Détaillée</b>                                         | <b>5</b>  |
| 2.1      | Extraction des étoiles et de leurs coordonnées . . . . .            | 5         |
| 2.1.1    | Objectifs . . . . .                                                 | 5         |
| 2.1.2    | Etat de l'art . . . . .                                             | 5         |
| 2.1.3    | Problèmes rencontrés et solutions choisies . . . . .                | 5         |
| 2.2      | Correspondance des constellations avec la base de données . . . . . | 7         |
| 2.2.1    | Objectifs . . . . .                                                 | 7         |
| 2.2.2    | Etat de l'art . . . . .                                             | 7         |
| 2.2.3    | Problèmes rencontrés et solutions choisies . . . . .                | 7         |
| 2.3      | Interface . . . . .                                                 | 10        |
| 2.3.1    | Objectifs . . . . .                                                 | 10        |
| 2.3.2    | Etat de l'art . . . . .                                             | 10        |
| 2.3.3    | Problèmes rencontrés et solutions choisies . . . . .                | 11        |
| <b>3</b> | <b>Communication &amp; Manuel</b>                                   | <b>13</b> |
| 3.1      | Etape du programme & Mode d'emploi utilisateur . . . . .            | 13        |
| 3.2      | Communications interprocessus . . . . .                             | 13        |
| 3.2.1    | Interface   Détecteur d'étoiles . . . . .                           | 13        |
| 3.2.2    | Détecteur d'étoiles   Reconnaissance . . . . .                      | 13        |
| 3.2.3    | Reconnaissance   Interface . . . . .                                | 13        |
| <b>4</b> | <b>Conclusion</b>                                                   | <b>14</b> |
| 4.1      | Retour sur les Tests Unitaires . . . . .                            | 14        |
| 4.2      | Comparaison entre l'objectif et la réalisation . . . . .            | 14        |
| <b>A</b> | <b>Documents complémentaires</b>                                    | <b>15</b> |

# 1 Introduction

## 1.1 Objectifs

Nous avons choisi de développer une **application dédiée à la reconnaissance automatique des constellations** dans le ciel nocturne. Des applications connues de reconnaissance de constellation existent déjà comme SkyView, cependant, elles utilisent uniquement la position de la constellation à une certaine période. Nous avons dès le début la volonté **d'utiliser des méthodes courantes chez les astronomes**. Ainsi, nous nous sommes orientés vers la reconnaissance automatique des constellations à partir **d'algorithmes d'analyse spatiale (pattern matching)** en utilisant l'outil préféré des astronomes : **La Géométrie**.

## 1.2 Contexte

### Astrométrie et Alignement

Lors de l'observation astronomique, il est **crucial de savoir précisément quelles parties du ciel sont observées**. Cela permet de corréler les observations avec des catalogues d'étoiles. Contrairement à de nombreux objets de la nature **les constellations**, du fait de leur éloignement conséquent, **ne se déforment pas selon la position depuis laquelle on les observe**. Seule leur luminosité varie et **on peut repérer avec précision la position des constellations en fonction de la période de l'année** (rotation de la Terre autour de son axe passant par l'étoile Polaire).

Les astronomes identifient les étoiles du ciel pour déterminer la position et l'orientation d'un télescope ou d'une caméra, **créer de cartes précises du ciel** ou encore **se repérer en mer grâce au ciel**. Les astronomes amateurs contribuent grandement à la recherche en Cosmologie. Leur aide pour l'observation de phénomènes répétitifs ou pour des surveillances systématiques en prévision de comètes ou de supernovae est essentielle tant les chercheurs manquent de temps pour les réaliser.

## 1.3 Plan & Recettes

Voici les étapes que nous avons suivi ainsi que leur recette :

### Extraction d'étoiles

**Extraire** les points lumineux d'une image,

**Retourner** les coordonnées précises et les caractéristiques des étoiles avec une liste d'adjacence.

### Correspondance avec la base de données

**Créer** des triangles à partir de points,

**Retourner** le nom de la constellation la plus similaire présente dans la base de données.

### Interface

**Afficher** l'image correspondante à la photographie dans la base de données ainsi qu'un texte original décrivant succinctement la constellation.

### 1.3.1 Reconnaissance Automatique de Constellations

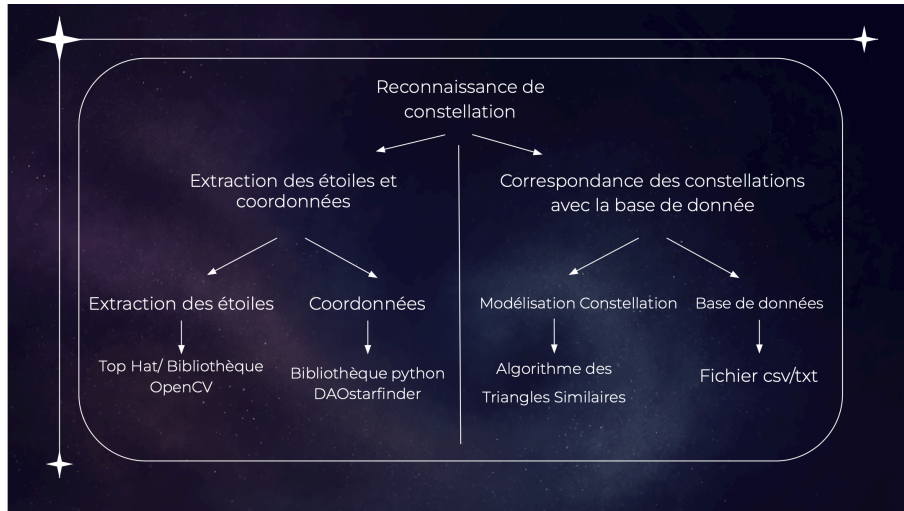


FIGURE 1 – Schéma d'organisation de la partie reconnaissance.

La partie algorithmique de reconnaissance de constellation est partagée par **deux langages** qui exploitent chacun leur plus grande force. En effet, **l'extraction d'étoiles et la manipulation d'image sont gérés par Python** et ses librairies tandis que **le corps du programme est en Java**. On profite de l'optimisation de Python pour les calculs et la structure de classes de Java. En réalité, suite à la création de l'interface, le projet est passé à une structure Maven et l'interface est devenue le centre de commande du logiciel.

### 1.3.2 Interface

D'abord en JavaSwing, l'interface a évolué en JavaFx pour des raisons de praticité et surtout car JavaSwing n'est plus mis à jour.

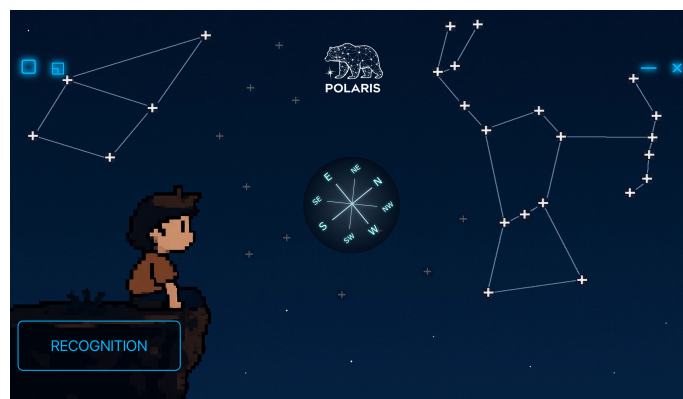


FIGURE 2 – Interface du logiciel *Polaris*

### 1.3.3 Diagramme de classes

Le diagramme de classe du projet contient les classes de la partie reconnaissance et de la partie interface. La partie Python ne contient pas de classes.

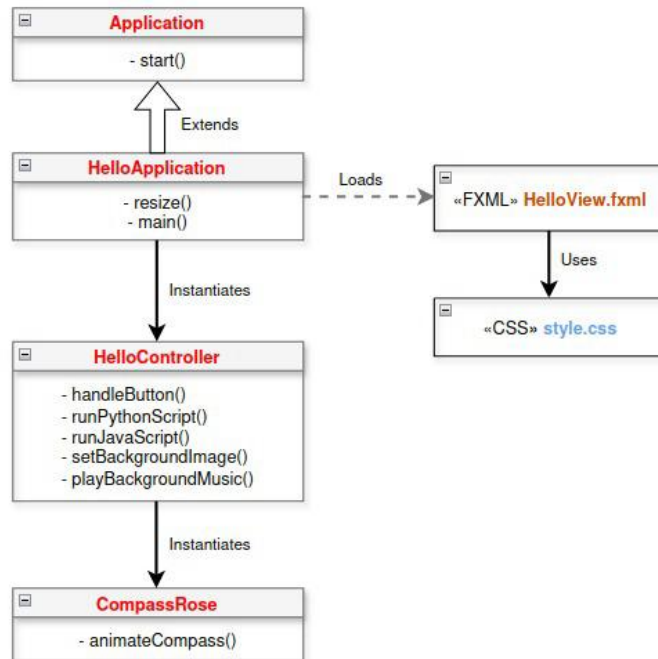


FIGURE 3 – Diagramme de l'interface (Apparaissent seulement les méthodes essentielles)

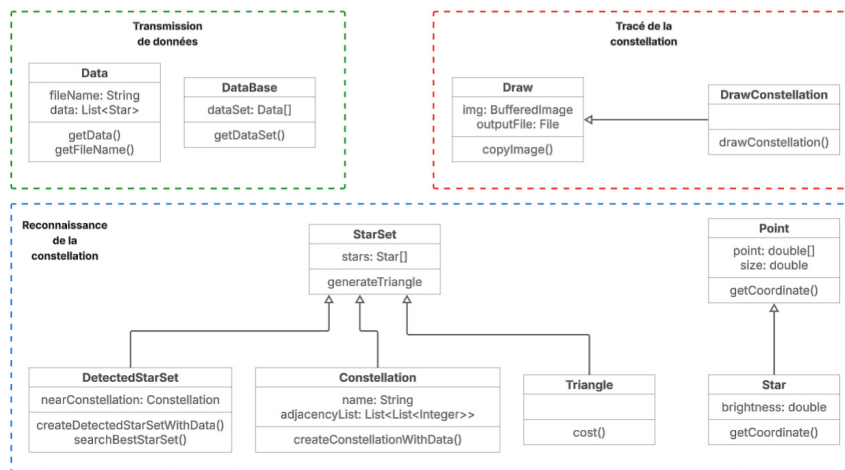


FIGURE 4 – Diagramme de la partie reconnaissance (Apparaissent seulement les méthodes essentielles)

## 1.4 Choix des Technologies

### Logiciel de version

**GitHub** pour la collaboration et la gestion du code source.

## Langages de Programmation

**Python** pour la détection d'étoile (traitement d'image).

**Bibliothèques** : DAOStarFindfer (Pillow), NumPy qui est une bibliothèque adaptée à notre problématique.

**Java** pour la reconnaissance de constellation et interface : rapide et performant.

**Bibliothèques** : JavaFx pour la praticité du fxml, les compatibilités CSS, etc.

## Tests Unitaires

**JUnit** : framework gratuit et open source pour les tests unitaires Java

**Unittest** : framework pour les tests unitaires Python

## IDE

**VScode** pour plus de flexibilité pour passer d'un langage à un autre

## Autres outils utilisés finalement abandonnés pour de meilleurs

**Bibliothèques** : JavaSwing pour apprendre les bases de l'interface via Java

**Format de transmission de données** : Json pour l'échange de données en texte lisible

**IDE** : Apache Netbeans pour l'interface Java Swing.

## 1.5 Gestion de projet

### Diagramme de Gantt

Le projet Polaris a rapidement trouvé ses 4 membres et l'équipe a donc pu commencer le brainstorming et la bibliographie dès fin septembre. Le plan de charge du projet pourra être retrouvé en annexe.

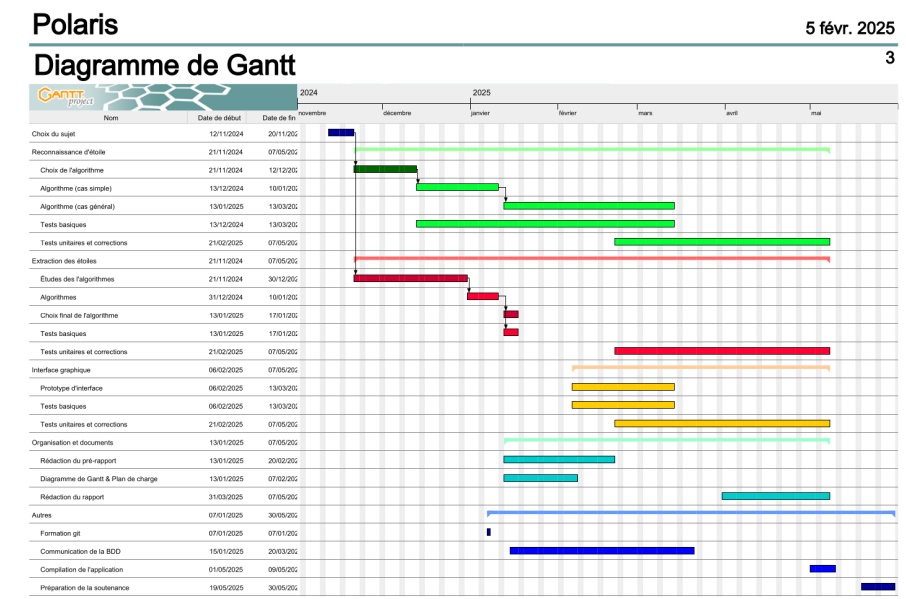


FIGURE 5 – Diagramme de Gantt

## 2 Conception Détaillée

### 2.1 Extraction des étoiles et de leurs coordonnées

#### 2.1.1 Objectifs

L'Algorithme doit être capable de reconnaître une étoile puis de nous fournir ses coordonnées sur l'image.

#### 2.1.2 Etat de l'art

Nous sommes dans un premier temps concentrés sur les différents algorithmes et méthodes déjà implémentées pour l'extraction d'étoiles et le traitement d'image.

#### SIFT (Scale-Invariant Feature Transform)

Méthode de détection et description de zones d'intérêt (features) où l'intensité lumineuse varie fortement. Le détecteur de Moravec analyse les variations d'intensité dans plusieurs directions autour d'un pixel. Une forte variation indique une feature, comme une étoile.

#### Top Hat pour l'extraction de points lumineux

Transformation morphologique utilisée pour extraire de petits détails brillants :

- *White top-hat* = image – ouverture (isole les zones plus claires),
- *Black top-hat* = fermeture – image (isole les zones plus sombres).

Utilisée pour améliorer l'image ou corriger l'arrière-plan.

#### DAOStarFinder

Algorithme de détection d'étoiles en trois étapes :

1. **Filtrage TopHat** : supprime le fond diffus pour isoler les objets brillants.
2. **Détection de pics lumineux** : recherche les pics ressemblant à des étoiles, filtre le bruit.
3. **Résultat** : liste d'étoiles avec coordonnées, brillance et taille.

#### 2.1.3 Problèmes rencontrés et solutions choisies

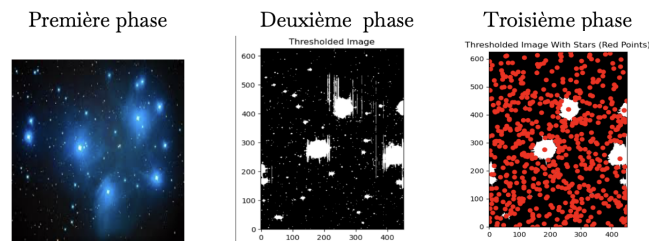


FIGURE 6 – DAOstarfinder et étoiles avec halos étendus

Néanmoins, lorsque nous avons choisi la bibliothèque Python DAOStarFinder, nous nous sommes rendu compte qu'elle détectait **un trop grand nombre d'étoiles**, y compris des faux positifs. En effet, elle avait du mal à bien délimiter les contours,

car certaines étoiles étaient **beaucoup plus lumineuses que d'autres**, ce qui créait des **halos plus étendus** autour d'elles et perturbait la détection. (Figure 6)

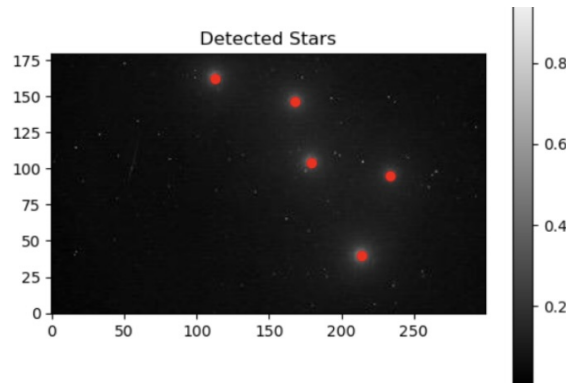


FIGURE 7 – DAOstarfinder avec étoiles bien définies

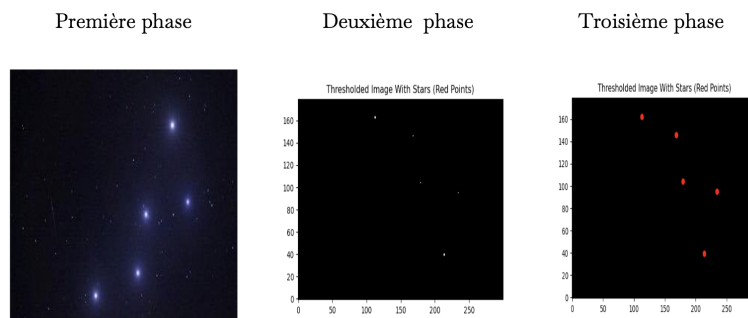


FIGURE 8 – détection étoiles via notre méthode

Dans le prolongement des difficultés précédemment rencontrées avec la détection d'étoiles, nous avons tiré parti de certaines fonctionnalités de la bibliothèque **DAOStarFinder**, en particulier du **seuil de détection** qu'elle génère par **indexation booléenne**.

Nous avons ensuite développé un script permettant d'exploiter ce masque binaire afin d'identifier les zones lumineuses correspondant aux étoiles, ainsi qu'à leurs halos.

Pour chaque forme détectée, nous avons calculé le **barycentre** afin d'estimer les **coordonnées centrales** de l'étoile associée.

À chaque étoile ainsi identifiée, nous avons attribué :

- une **coordonnée** (déterminée à l'aide de notre propre méthode de calcul de barycentre),
- une **valeur de luminosité** (obtenue via la bibliothèque **Pillow**, en analysant l'intensité des pixels),
- et une **taille apparente**, estimée à l'aide d'un algorithme que nous avons conçu spécifiquement.

## 2.2 Correspondance des constellations avec la base de données

### 2.2.1 Objectifs

Utiliser les propriétés d'invariance des constellations pour les différencier, Comparer les listes d'étoiles formées avec les constellations de la base de donnée avec une fonction de coût et Retourner le nom de la constellation correspondant le plus.

### 2.2.2 Etat de l'art

Pour la correspondance avec la base de données, la solution de l'IA étant trop nécessiteuse en données et pas forcément adaptée à notre cas, nous avons décidé de calculer une fonction de coût qui permet de distinguer les caractéristiques des constellations.

#### ICP (Iterative Closest Point)

Algorithme d'appariement de nuages de points :

1. Associe chaque point source à son plus proche voisin dans le nuage cible.
2. Calcule la transformation (rotation + translation) minimisant la distance.
3. Applique la transformation au nuage source.
4. Répète jusqu'à convergence (écart minimal entre itérations).

#### Algorithme des Triangles similaires

Compare des motifs géométriques formés par des groupes d'étoiles et exploite l'invariance des proportions des triangles.

**1. Triangulation** : On forme des triangles à partir des étoiles visibles dans l'image capturée (par exemple, à partir des plus brillantes).

**2. Caractérisation** : Chaque triangle est défini par des ratios de distances et ses angles.

**3. Base de données** : Ces caractéristiques sont comparées à une base de données contenant des triangles pré-calculés à partir de catalogues d'étoiles connues.

**4. Correspondance** : Si une correspondance est trouvée, elle minimise la fonction de coût du triangle.

### 2.2.3 Problèmes rencontrés et solutions choisies

3 questions essentielles se sont posées lors de notre réflexion sur la partie reconnaissance :

- Quels critères pertinents utiliser pour distinguer deux constellations ?
- Aussi, comment comparer la liste d'étoiles de l'image avec les constellations de la base de donnée à partir de ces critères ?
- Enfin, comment dessiner la constellation une fois la constellation identifiée ?

**Grandeurs invariantes d'un triangle, nos critères ?**

Puisque notre démarche de reconnaissance de constellation s’inspire de celle des astronomes, l’algorithme des triangles similaires était un choix qui nous a semblé judicieux. Les grandeurs propres et invariantes (par translation, rotation, changement d’échelle) d’un triangle sont notamment ses angles et ses rapports de longueur.

### Fonction de coût

Nous avons choisi de normaliser la somme des rapports des côtés du triangle (rapports préalablement triés en amont entre les longueurs des côtés) et des angles de chaque triangle. En effet, les deux caractéristiques (rapports de longueurs et angles) ont la même importance donc nous leur attribuons un poids égal. Pour cela il faut les mettre au même ordre de grandeur (entre 0 et 1) d’où la normalisation.

### Détection des étoiles

#### 1. Choix méthodologiques

Le système de détection repose sur une approche simple mais plutôt robuste, visant à **extraire les étoiles les plus significatives** (les plus brillantes) d’une image astronomique. L’objectif est de réduire l’impact du bruit et de simplifier la correspondance avec les constellations.

#### 2. Méthodologie

Extraction des étoiles brillantes :

Analyse de l’image pour détecter les sources lumineuses. **Sélection des 20 étoiles les plus brillantes** qui est une instance de `DetectedStarSet` (luminosités comprises dans les informations fournies par le script Python dans le CSV).

Correspondance avec la constellation cible - Part I :

Pour chaque **Constellation**, seules les 4 étoiles les plus brillantes qui correspondent le mieux à une certaine constellation sont retenues. Si la constellation possède plus d’étoiles, on calculera leur position avec de la géométrie (abordée plus tard). Cette sélection diminue la fiabilité de la reconnaissance mais diminue de manière significative le temps de calcul.

Correspondance avec la constellation cible - Part II :

La fonction de coût est définie pour mesurer la proximité (géométrique et photométrique) entre ces 4 étoiles et un sous-ensemble équivalent parmi les étoiles détectées. On regarde toutes les combinaisons de 4 étoiles et nous conservons celle qui minimise le coût.

#### 3. Éléments majeurs du système

**Star** : Représente une étoile individuelle avec ses coordonnées (x, y) dans l’image et sa brillance. C’est l’unité de base du traitement.

**StarSet** : Représente un ensemble d’étoiles composée de plusieurs étoiles. Permet de générer des triangles à partir des points.

**DetectedStarSet** : Représente une liste d’étoiles dans l’image à analyser. Cette classe permet notamment de générer des combinaisons d’étoiles et d’identifier l’ensemble d’étoiles correspondant le mieux à une constellation. Offre des méthodes qui forment des triangles et extraient des sous-ensembles candidats.

**Triangle** : Représente un triplet d’étoiles. Chaque triangle est caractérisé par ses longueurs de côtés et ses angles internes. Il sert à comparer des structures stellaires

(patrons) entre l'image et les constellations.

**Constellation** : Contient une description d'un groupe d'étoiles connues. Inclut un nom, une liste d'adjacence qui représente schématiquement les liaisons entre étoiles et de la même façon que DetectedStarSet, on peut calculer les triangles formés par les étoiles principales.

#### 4. Problème d'hypothèse et de complexité

L'hypothèse initiale pour le calcul du coût des triangles : l'ordre des étoiles est conservé pour un même set d'étoiles par rapport à celui de la base de données. Seulement, cette hypothèse n'est pas toujours respectée lorsque l'on fournit une image différente de la même constellation qui a permis de faire la base de données et les coûts calculés sont donc faux. Il faudrait donc maintenant itérer sur toutes les permutations d'étoiles. D'où le problème de complexité.

L'extraction des étoiles nous permet de ranger les étoiles d'une images dans leur ordre de brillance.



FIGURE 9 – Exemple de classement

Par exemple, à gauche on a une image à analyser avec des étoiles, après extraction, on peut numéroter les étoiles en fonction de leur brillance (figure de droite). Ainsi, nous avons au début fait l'hypothèse que ce classement était toujours le même, quelle que soit la manière dont on prenait en photo une constellation. Cette hypothèse nous simplifiait grandement le calcul de la correspondance entre une constellation et un set de d'étoiles. Malheureusement, après de nombreux tests, nous nous sommes rendus compte qu'elle était dans beaucoup de cas pas vérifiée.

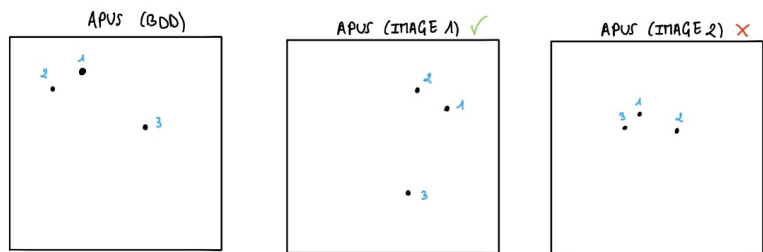


FIGURE 10 – Exemple de contradiction de l'hypothèse

Voici un exemple avec la constellation Apus. Sur la gauche, on observe la constellation telle qu'elle a été considérée dans la base de données. Au milieu, une image de la même constellation prise à un autre moment, où les étoiles ont été extraites et

où la condition est vérifiée. Enfin, sur la droite, une image où la condition n'est pas vérifiée. Dans le cas où les rangs ne sont pas conservés, le coût entre la constellation et le set d'étoiles sera faussé.

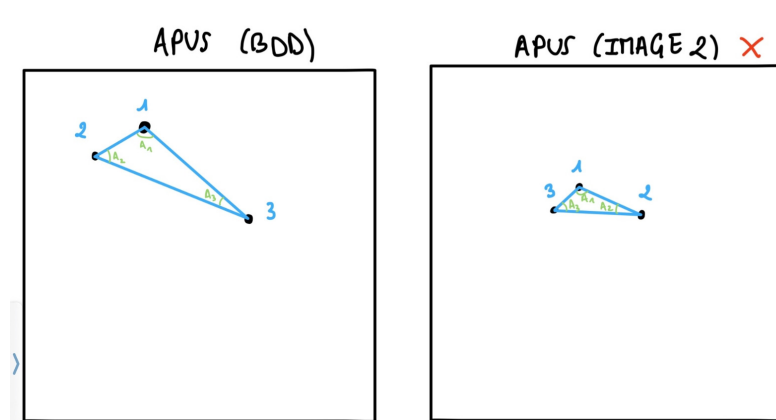


FIGURE 11 – Exemple de cas où le coût est anormalement élevé pour la même constellation

En reprenant l'exemple précédent, on observe que les angles 2 et 3 ne correspondent pas. (On peut faire la même conclusion pour les longueurs normalisées) Cela va donc provoquer un grand coût, alors que c'est la bonne constellation. Il y aura donc des erreurs de reconnaissance. Ainsi, nous avons abandonné cette hypothèse. Mais cela signifiait qu'il fallait énumérer toutes les possibilités de rangs entre les étoiles et choisir uniquement celle qui minimisait le coût. Notre algorithme présentait déjà des temps de calculs significatifs (autours de la minute), et ce changement de paradigme a amplifié ce phénomène. Après réflexion, nous nous sommes tournées vers la solution suivante : Plutôt que de minimiser le coût entre un ensemble complet d'étoiles et une constellation, nous avons restreint notre recherche à la comparaison entre tous les groupes de 4 étoiles de l'image et les 4 étoiles les plus brillantes de chaque constellation. Une fois la meilleure correspondance identifiée, nous avons utilisé des propriétés géométriques en particulier les angles et les positions relatives pour localiser les autres étoiles de la constellation sur l'image.

## 2.3 Interface

### 2.3.1 Objectifs

Réaliser une interface qui lorsqu'on lui donne une image du ciel nous renvoie la constellation associée si une constellation est présente.

### 2.3.2 Etat de l'art

Dans le développement d'interfaces graphiques en Java, deux bibliothèques principales coexistent : **Java Swing** et **JavaFX**.

— **Java Swing**

Swing a longtemps été la bibliothèque standard pour créer des interfaces graphiques en Java. Elle offre une large panoplie de composants (boutons, fenêtres, menus...) et repose sur le concept de **modèles de rendu léger**. Cependant, **Swing est aujourd'hui considéré comme obsolète** pour plusieurs raisons :

- Son architecture est datée, moins réactive et peu adaptée aux applications modernes riches en interactions et animations.
  - Le design des composants est rigide et difficile à personnaliser sans manipulations complexes.
  - Elle n'est plus activement développée : bien qu'encore maintenue, elle n'évolue plus vraiment.
- **JavaFX**

JavaFX est le successeur officiel de Swing. Il a été conçu pour répondre aux **besoins des interfaces modernes** : plus dynamiques, plus fluides, et plus esthétiques. JavaFX apporte plusieurs avantages notables :

- **Gestion améliorée des événements** et du **multimédia** (audio, vidéo, animations).
- Meilleure intégration des **effets graphiques**, des **animations**, et des **styles CSS**.
- Possibilité de créer des interfaces plus réactives, évolutives, et orientées utilisateur.
- Il prend également en charge des outils de conception visuelle comme **Scene Builder**.

**Pourquoi nous avons choisi JavaFX** Le choix de **JavaFX** s'est imposé naturellement dans le cadre de notre projet. Notre interface nécessitait :

- une **grande flexibilité graphique** (pour afficher des éléments dynamiques comme la rose des vents, les points lumineux, etc.),
- une **gestion fluide des événements** (animations, interactions utilisateur)

JavaFX s'est révélé plus adapté que Swing sur tous ces points, en particulier grâce à son **système de scène**, sa prise en charge native des **transitions animées**, et sa capacité à **lier dynamiquement les propriétés** des composants à des données ou à des calculs.

### 2.3.3 Problèmes rencontrés et solutions choisies

Au début de notre développement, nous avons opté pour **Scene Builder** afin de faciliter la création de notre interface. Cependant, après plusieurs essais, nous avons constaté que la **prise en main** s'est révélée plus complexe que prévu, notamment quand il s'agissait de placer correctement les éléments sur la scène. En effet, certains boutons ne s'affichaient pas à l'emplacement souhaité une fois le programme lancé.

Nous avons donc pris la décision de **ne pas utiliser Scene Builder** et de passer directement par un fichier **FXML** pour définir la structure de notre interface, et un fichier **CSS** pour styliser les composants. Cette approche a été plus **simple** à maîtriser, mais elle a tout de même présenté des **difficultés**.

En particulier, la gestion du placement des éléments s'est avérée délicate : les composants **s'influençaient les uns les autres** (notamment avec les **layouts** comme **VBox** ou **HBox**), ce qui a rendu complexe le positionnement précis des éléments. De plus, l'**ordre des éléments dans le fichier FXML** a joué un rôle crucial, car il affectait leur positionnement et leur affichage à l'écran.

De plus, il est important de noter que **JavaFX** n'est pas entièrement compatible avec certaines propriétés du **CSS**, ce qui a compliqué la personnalisation de certains éléments visuels. En particulier, pour le **loader** que nous voulions intégrer, il a été nécessaire de **créer une classe Java dédiée** pour implémenter sa fonctionnalité et son animation, car les styles **CSS** ne permettaient pas de réaliser cette animation complexe.

## 3 Communication & Manuel

### 3.1 Etape du programme & Mode d'emploi utilisateur

Notre application fonctionne en 4 étapes :

- 1- Sélection de l'image à traiter.
- 2- Extraction des étoiles présentes sur l'image
- 3- Reconnaissance de la constellation avec le set d'étoiles
- 4- Affichage de la constellation et de son histoire

Naturellement, il est nécessaire de faire communiquer tous les programmes responsables, surtout que notre projet est multilingage (Java et Python).

L'utilisateur lui, n'a que trois étapes à suivre :

- 1- Cliquer sur Recognition pour sélectionner l'image de son choix
- 2 - Une fois le chargement fini (repéré par la disparition de la boussole centrale), il peut cliquer sur le bouton Constellation qui apparaît aussitôt
- 3- La Constellation apparaît et un texte avec, il peut fermer le logiciel ou recliquer sur Constellation pour revenir à l'écran principal

### 3.2 Communications interprocessus

#### 3.2.1 Interface | Détecteur d'étoiles

**Java -> Python : Fichier txt**

La première étape de fonctionnement de l'application est la sélection de la photo du ciel étoilé. L'interface doit donc communiquer l'emplacement de l'image pour qu'elle soit traitée par le détecteur d'étoiles. Pour cela, on écrit le path de l'image choisi (via l'interface) dans un fichier txt nommé output.txt.

#### 3.2.2 Détecteur d'étoiles | Reconnaissance

**Python -> Java : Fichier CSV**

La seconde étape est le traitement de l'image et l'extraction des étoiles. Pour fournir leur localisation à la partie reconnaissance, nous avons décidé, après des tests infructueux avec le format json, d'utiliser le format **csv**.

#### 3.2.3 Reconnaissance | Interface

**Java -> Java : Fichier txt**

Tout comme le passage de la première à la seconde étape, on peut facilement de communiquer le nom de la constellation reconnue. L'idée est encore une fois d'écrire le nom dans un **.txt**.

## 4 Conclusion

### 4.1 Retour sur les Tests Unitaires

- **Couverture des tests Java :**

Méthodes qui présentaient le plus de difficultés algorithmiques et méthodes simples les plus utilisées testées : Couverture quasi-complète. Cas d'usage couverts : cas limites, gestion des erreurs.

- **Couverture des tests Python :**

Fonctions principales. Couverture quasi-complète. Cas d'usage couverts : cas limites, gestion des erreurs.

- **Résultats :**

La majorité des tests sont passés avec succès. De nombreux bugs détectés grâce aux Tests Unitaires (dénombrement pour la formation des sous groupes d'étoiles). Corrections apportées après détection (changement d'un indice et import d'un module spécialisé).

- **Limites :**

Certains modules difficiles à tester (ex. : interfaces graphiques).

- **Enseignements tirés :**

Ecrire les tests en parallèle du code est crucial pour ne pas se retrouver dans une situation compliquée. Les tests ont facilité le débogage, amélioré la robustesse du code. L'automatisation des tests permet des vérifications rapides et continues lors des modifications.

### 4.2 Comparaison entre l'objectif et la réalisation

L'objectif initial a été atteint avec succès, et nous sommes d'autant plus fiers de notre réalisation qu'elle s'est révélée ambitieuse — il s'agissait en effet du tout premier projet de développement informatique pour l'ensemble de l'équipe. Si certaines idées ont dû être abandonnées en cours de route, nous avons néanmoins réussi à intégrer toutes les fonctionnalités essentielles, tout en conservant une identité visuelle cohérente grâce à notre interface graphique. Ce projet s'est avéré extrêmement formateur à bien des égards : il nous a permis de découvrir la richesse des langages de programmation, qu'ils soient orientés objet ou procéduraux, de concevoir des interfaces graphiques complètes, et de nous initier aux sciences dures. L'astronomie, un domaine qui nous passionne particulièrement, a constitué le cœur de notre travail. Bien que des solutions plus abouties existent déjà, nous sommes fiers de présenter la nôtre : une modeste contribution à ce domaine fascinant.

## A Documents complémentaires

| SUIVI D'ACTIVITÉS (Charge Consommée) |
|--------------------------------------|
|--------------------------------------|

| PLAN DE CHARGES PRÉVISIONNEL |
|------------------------------|
|------------------------------|

| Description de l'activité                  | Charge en % | Charge en H | Charge en H / Participant |                  |                          |    | Ryane SERI |
|--------------------------------------------|-------------|-------------|---------------------------|------------------|--------------------------|----|------------|
|                                            |             |             | Emma MELL                 | Chadi AIT EKIOUI | Beryl SPERELAKIS-BEEDHAM |    |            |
| Total                                      | 100%        | 200         | 50                        | 50               | 50                       | 50 | 50         |
| Gestion de projets                         | 28%         | 56          | 14                        | 14               | 14                       | 14 | 14         |
| Réunion de lancement                       | 2%          | 4           | 1                         | 1                | 1                        | 1  | 1          |
| Planning prévisionnel et Suivi d'activités | 2%          | 4           | 1                         | 1                | 1                        | 1  | 1          |
| Réunions de suivi                          | 12%         | 24          | 6                         | 6                | 6                        | 6  | 6          |
| Rédaction                                  | 8%          | 16          | 4                         | 4                | 4                        | 4  | 4          |
| Outils collaboratifs (glt, etc.)           | 4%          | 8           | 2                         | 2                | 2                        | 2  | 2          |

|                                |    |   |   |   |   |   |   |
|--------------------------------|----|---|---|---|---|---|---|
| Spécification                  | 4% | 8 | 2 | 2 | 2 | 2 | 2 |
| Définition des fonctionnalités | 4% | 8 | 2 | 2 | 2 | 2 | 2 |

|                                                                 |     |    |   |   |   |   |   |
|-----------------------------------------------------------------|-----|----|---|---|---|---|---|
| Conception préliminaire                                         | 12% | 24 | 6 | 6 | 6 | 6 | 6 |
| Définition d'un modèle de données                               | 2%  | 4  | 1 | 1 | 1 | 1 | 1 |
| Définition d'un format de fichiers associé au modèle de données | 2%  | 4  | 1 | 1 | 1 | 1 | 1 |
| Définition des fonctionnalités                                  | 6%  | 12 | 3 | 3 | 3 | 3 | 3 |
| Définition des modules                                          | 2%  | 4  | 1 | 1 | 1 | 1 | 1 |

|                                                |     |    |    |    |    |    |    |
|------------------------------------------------|-----|----|----|----|----|----|----|
| Conception détaillée                           | 22% | 44 | 11 | 11 | 11 | 11 | 11 |
| Définition des classes                         | 2%  | 4  | 1  | 1  | 1  | 1  | 1  |
| Définition des méthodes                        | 6%  | 12 | 3  | 3  | 3  | 3  | 3  |
| Définition des tests unitaires                 | 6%  | 12 | 3  | 3  | 3  | 3  | 3  |
| Auto-formation                                 | 4%  | 8  | 2  | 2  | 2  | 2  | 2  |
| Design interface (css, fxml, vidéo de fond...) | 4%  | 8  | 2  | 2  | 2  | 2  | 2  |

|                            |     |    |    |    |    |    |    |
|----------------------------|-----|----|----|----|----|----|----|
| Codage                     | 19% | 37 | 10 | 10 | 10 | 10 | 10 |
| Codage des classes         | 2%  | 4  | 1  | 1  | 1  | 1  | 1  |
| Codage des méthodes        | 11% | 21 | 6  | 6  | 6  | 6  | 6  |
| Codage des tests unitaires | 6%  | 12 | 3  | 3  | 3  | 3  | 3  |
| Débuggage                  |     |    |    |    |    |    |    |

|                          |    |    |   |   |   |   |   |
|--------------------------|----|----|---|---|---|---|---|
| Intégration              | 6% | 12 | 3 | 3 | 3 | 3 | 3 |
| Création base de données |    |    |   |   |   |   |   |
| Intégration des modules  | 4% | 8  | 2 | 2 | 2 | 2 | 2 |
| Tests d'intégration      | 2% | 4  | 1 | 1 | 1 | 1 | 1 |

|                              |    |    |   |   |   |   |   |
|------------------------------|----|----|---|---|---|---|---|
| Soutenance                   | 8% | 16 | 4 | 4 | 4 | 4 | 4 |
| Préparation de la soutenance | 6% | 12 | 3 | 3 | 3 | 3 | 3 |
| Soutenance                   | 2% | 4  | 1 | 1 | 1 | 1 | 1 |

| Charge en % | Charge en H | Charge en H / Participant |                  |                          |     | Ryane SERI |
|-------------|-------------|---------------------------|------------------|--------------------------|-----|------------|
|             |             | Emma MELL                 | Chadi AIT EKIOUI | Beryl SPERELAKIS-BEEDHAM |     |            |
| 237%        | 473         | 127                       | 119              | 112                      | 115 | 115        |
| 12%         | 55          | 10                        | 18               | 15                       | 12  | 12         |
| 1%          | 4           | 1                         | 1                | 1                        | 1   | 1          |
| 1%          | 4           | 1                         | 1                | 1                        | 1   | 1          |
| 5%          | 24          | 6                         | 6                | 6                        | 6   | 6          |
| 3%          | 15          | 0                         | 8                | 5                        | 2   | 2          |
| 2%          | 8           | 2                         | 2                | 2                        | 2   | 2          |
| 10%         | 48          | 12                        | 12               | 12                       | 12  | 12         |
| 10%         | 48          | 12                        | 12               | 12                       | 12  | 12         |
| 6%          | 27          | 5                         | 9                | 5                        | 8   | 8          |
| 1%          | 5           | 1                         | 2                | 1                        | 1   | 1          |
| 1%          | 6           | 0                         | 3                | 0                        | 3   | 3          |
| 3%          | 12          | 3                         | 3                | 3                        | 3   | 3          |
| 1%          | 4           | 1                         | 1                | 1                        | 1   | 1          |
| 21%         | 97          | 15                        | 23               | 34                       | 25  | 25         |
| 1%          | 5           | 2                         | 1                | 1                        | 1   | 1          |
| 4%          | 17          | 6                         | 5                | 3                        | 3   | 3          |
| 2%          | 10          | 2                         | 3                | 3                        | 2   | 2          |
| 7%          | 34          | 5                         | 10               | 10                       | 9   | 9          |
| 7%          | 31          | 0                         | 4                | 17                       | 10  | 10         |
| 11%         | 50          | 69                        | 43               | 38                       | 42  | 42         |
| 4%          | 18          | 6                         | 5                | 5                        | 2   | 2          |
| 4%          | 21          | 40                        | 25               | 20                       | 28  | 28         |
| 2%          | 11          | 3                         | 3                | 3                        | 2   | 2          |
|             |             | 20                        | 10               | 10                       | 10  | 10         |
| 3%          | 16          | 12                        | 10               | 4                        | 12  | 12         |
| 3%          | 16          | 2                         | 6                | 2                        | 6   | 6          |
| 0%          | 0           | 0                         | 0                | 0                        | 0   | 0          |
| 3%          | 16          | 4                         | 4                | 4                        | 4   | 4          |
| 3%          | 12          | 3                         | 3                | 3                        | 3   | 3          |
| 1%          | 4           | 1                         | 1                | 1                        | 1   | 1          |

Tâches

| Nom                                 | Date de début | Date de fin |
|-------------------------------------|---------------|-------------|
| Choix du sujet                      | 12/11/2024    | 20/11/2024  |
| Reconnaissance d'étoile             | 21/11/2024    | 07/05/2025  |
| Choix de l'algorithme               | 21/11/2024    | 12/12/2024  |
| Algorithme (cas simple)             | 13/12/2024    | 10/01/2025  |
| Algorithme (cas général)            | 13/01/2025    | 13/03/2025  |
| Tests basiques                      | 13/12/2024    | 13/03/2025  |
| Tests unitaires et corrections      | 21/02/2025    | 07/05/2025  |
| Extraction des étoiles              | 21/11/2024    | 07/05/2025  |
| Études des l'algorithmes            | 21/11/2024    | 30/12/2024  |
| Algorithmes                         | 31/12/2024    | 10/01/2025  |
| Choix final de l'algorithme         | 13/01/2025    | 17/01/2025  |
| Tests basiques                      | 13/01/2025    | 17/01/2025  |
| Tests unitaires et corrections      | 21/02/2025    | 07/05/2025  |
| Interface graphique                 | 06/02/2025    | 07/05/2025  |
| Prototype d'interface               | 06/02/2025    | 13/03/2025  |
| Tests basiques                      | 06/02/2025    | 13/03/2025  |
| Tests unitaires et corrections      | 21/02/2025    | 07/05/2025  |
| Organisation et documents           | 13/01/2025    | 07/05/2025  |
| Rédaction du pré-rapport            | 13/01/2025    | 20/02/2025  |
| Diagramme de Gantt & Plan de charge | 13/01/2025    | 07/02/2025  |
| Rédaction du rapport                | 31/03/2025    | 07/05/2025  |
| Autres                              | 07/01/2025    | 30/05/2025  |
| Formation git                       | 07/01/2025    | 07/01/2025  |
| Communication de la BDD             | 15/01/2025    | 20/03/2025  |
| Compilation de l'application        | 01/05/2025    | 09/05/2025  |
| Préparation de la soutenance        | 19/05/2025    | 30/05/2025  |

